EEG_API_Unicorn

Guide d'intégration : connecter une application, entraîner les modèles, recevoir les décisions

Version du 30 octobre 2026

Table des matières

1 Présentation 3
1.1 Les modes 3
1.2 LSL ou UDP 3
1.3 Ce que contient ce guide 3
2 Préparer le poste 5
2.1 Installer et lancer la console 5
2.2 Le réseau 5
2.3 Avant chaque séance avec le casque 5
3 Connecter une application 6
3.1 Choisir un client 6
3.2 Ce qui se passe 6
3.3 Les clients LSL : Python, Unity, MATLAB 7
3.4 Les clients UDP sur un ordinateur : Raspberry Pi, ROS 2 8
3.5 Les microcontrôleurs : ESP32, Arduino UNO R4 WiFi 9
4 Les réglages du mode 11
5 Les modèles 12
5.1 Entraîner un modèle 12
5.2 Tester un modèle 13
5.3 Les modèles pendant une connexion 13
5.4 Ce que le modèle impose 13
6 Afficher le stimulus : P300, ErrP, c-VEP 14
7 Ce que reçoit l'application 15
7.1 Les décisions, mode par mode 15
7.2 L'état du moteur 15
8 Écrire son propre client 16
8.1 Les commandes 16
8.2 Par LSL 16
8.3 Par UDP 17
9 Dépannage 19
10 Limites 20

1 Présentation

EEG_API_Unicorn lit un casque EEG **Unicorn Hybrid Black**, décode le signal et publie le résultat sur le réseau local. Votre application (un jeu, une visualisation, un robot, des LED…) s'y connecte, choisit un mode et ses réglages, puis reçoit les décisions.

```
casque Unicorn ──Bluetooth──▶  la CONSOLE (sur un PC)  ──réseau──▶  VOTRE APPLICATION
(8 électrodes)                 - lit le casque                       - envoie ses réglages
                               - entraîne les modèles                - démarre un mode
                               - décode                              - reçoit les décisions
                               - publie les décisions                - agit (jeu, LED, robot…)
```

- **La console** est le programme qui tient le casque. Tout s'y fait par boutons : vérifier le casque, entraîner un modèle, le tester, connecter une application.
- **Votre application ne touche jamais le casque.** Elle parle à la console par le réseau, même quand elle tourne sur le même PC.
- **Une décision est une intention neutre** : « la cible 1 est regardée », « la tâche LEFT est imaginée ». Jamais « avancer le robot » : c'est votre application qui en fait une action.

1.1 Les modes

Mode	Ce qu'il décode	Modèle	Ce que fait votre application	Par UDP
SSVEP	la cible regardée, parmi des cibles qui clignotent chacune à sa fréquence	non	fait clignoter les cibles (à l'écran ou sur des LED)	oui
Motor Imagery (MI)	un mouvement imaginé : une main, les pieds…	oui	rien de plus	oui
Neuro	un état mental : charge, somnolence, engagement	non	rien de plus	oui
P300	la cible choisie, parmi des cibles qui flashent tour à tour	oui	fait flasher les cibles, envoie un marqueur à chaque flash	non
ErrP	la réaction du cerveau à une erreur de la machine	oui	envoie un marqueur à chaque résultat affiché	non
c-VEP	la cible regardée, parmi des cibles qui clignotent selon un code	oui	affiche le code image par image, envoie une horloge	non
Brut (`raw`)	rien : les 8 voies EEG telles quelles, à 250 Hz	non	rien de plus	non

Le **SSVEP** est le plus simple pour commencer : il n'a ni modèle à entraîner, ni marqueurs à envoyer.

1.2 LSL ou UDP

Votre application parle à la console de l'une de ces deux façons :

	LSL	UDP
Pour	une application sur un ordinateur : Python, Unity, MATLAB…	un appareil sans LSL : ESP32, Arduino, Raspberry Pi, ROS 2
Modes	tous	SSVEP, Motor Imagery, Neuro
Côté moteur	la console, ou le moteur seul (`server.py`)	la console seulement
Décisions	lues sur le flux du mode	envoyées par la console à l'appareil

LSL (*Lab Streaming Layer*) est une bibliothèque qui transporte des flux de données sur le réseau local. **UDP** est le protocole réseau le plus simple : un paquet envoyé à une adresse, sans connexion préalable. Les commandes et les réponses sont les mêmes des deux côtés.

1.3 Ce que contient ce guide

- Les parties 2 à 5 suffisent pour connecter une application avec un **client prêt à l'emploi** : il n'y a qu'un fichier de configuration à remplir.

- La partie 6 concerne les modes où votre application affiche elle-même le stimulus (P300, ErrP, c-VEP), la partie 7 ce que contient une décision.
- La partie 8 décrit le protocole, pour écrire un client sur une autre plateforme.

Documents de référence, en anglais : `docs/control.md` (les commandes, les réponses, le protocole UDP), `docs/markers.md` (les marqueurs), `docs/network.md` (le réseau et le pare-feu), et le `README.md` de chaque dossier de `clients/`.

2 Préparer le poste

2.1 Installer et lancer la console

Avec l'installeur (Windows 64 bits, sans Python) :

1. Téléchargez `EEG-API-Unicorn-Setup-<version>.exe` sur la page Releases de https://github.com/medkar/EEG_API_Unicorn, puis lancez-le.
2. L'installeur n'est pas signé : Windows SmartScreen affiche un avertissement la première fois. Cliquez sur **Informations complémentaires**, puis **Exécuter quand même**.
3. Choisissez **Installer pour tous les utilisateurs** si vous le pouvez : l'installeur ajoute alors la règle du pare-feu de Windows (voir *Le réseau*).

Lancez ensuite **EEG API Unicorn** depuis le menu Démarrer.

Depuis les sources (Python 3.12 ou 3.13) :

```
pip install -r requirements.txt      # une fois
outils\Console EEG.bat                # ou un double-clic dans l'explorateur
```

Au démarrage, la console pose deux questions :

Question	Réponse
La source de la séance	le casque Unicorn (par son numéro de série), ou le **board de test** : un signal synthétique, pour développer sans casque
Le secteur électrique	50 Hz en Europe, 60 Hz en Amérique

Elle ne passe jamais d'elle-même sur le board de test : si le casque refuse de s'ouvrir, elle repose la question.

Langue. La console s'ouvre en anglais. Ce guide cite ses libellés en français : cliquez sur « 🌐 », en haut à droite de l'accueil, pour la passer en français. Les données envoyées à votre application restent en anglais, quelle que soit la langue affichée.

2.2 Le réseau

- Votre application et la console sont sur le **même réseau local**, dans le même sous-réseau.
- **Pare-feu de Windows** : sur le PC de la console, autorisez **EEG API Unicorn** en entrée (ou Python, si la console tourne depuis les sources), réseaux publics compris. Installé pour tous les utilisateurs, la règle est déjà là ; sinon, Windows le demande à la première utilisation. Pour une application LSL sur un autre PC, autorisez aussi ce programme (Python, Unity…) sur son PC.
- Certains réseaux Wi-Fi **isolent** les appareils les uns des autres. Le plus simple est alors le **point d'accès mobile** du PC de la console : Paramètres → Réseau et Internet → Point d'accès sans fil mobile, en 2,4 GHz pour un microcontrôleur. Le PC y a l'adresse `192.168.137.1`.

Le détail est dans `docs/network.md`.

2.3 Avant chaque séance avec le casque

1. **Humidifiez les électrodes au sérum physiologique** : c'est le premier facteur de qualité du signal.
2. Sur l'accueil, lancez « **Vérifier le casque** » (37 s). La page montre le contact de chaque électrode (vert : bon, rouge : mauvais), puis mesure le rythme alpha, yeux ouverts puis fermés. Si elle affiche « **ARRÊTE ICI** », replacez les électrodes et recommencez : aucune mesure ne serait fiable.
3. **Gardez la console ouverte toute la séance.** À sa réouverture, l'amplificateur du casque redémarre et deux voies saturent un moment.

3 Connecter une application

3.1 Choisir un client

Le dossier `clients/` contient un client prêt à l'emploi par plateforme. Chacun fait toute la connexion à votre place. Vous ne touchez qu'à deux choses :

- **son fichier de configuration** : le nom de l'application, le mode, ses réglages, le réseau ;
- **son gestionnaire de décisions** : la fonction qui reçoit chaque décision, où va votre code.

La bibliothèque, elle, s'utilise telle quelle. Copiez le dossier dans votre projet, remplissez la configuration, puis remplacez le gestionnaire de décisions.

Dossier	Plateforme	Transport	Configuration	Votre code
`clients/python-lsl/`	Python 3 sur un ordinateur	LSL	`config.json`	`on_decision()` dans `example.py`
`clients/unity/`	Unity 6	LSL	`StreamingAssets/eeg_api_config.json`	`onTargetChanged` ou `DecisionReceived`
`clients/matlab/`	MATLAB R2016b ou ultérieur	LSL	`config.json`	`onDecision()` dans `example.m`
`clients/raspberry-pi/`	Python 3, bibliothèque standard seule : Raspberry Pi, carte Linux	UDP	`config.json`	`on_decision()` dans `example.py`
`clients/ros2/`	ROS 2	UDP	`config/params.yaml`	un nœud abonné à `eeg/target`
`clients/esp32/`	ESP32 (Arduino)	UDP, Wi-Fi	`config.h`	`onDecision()` dans le croquis
`clients/arduino-wifi/`	Arduino UNO R4 WiFi	UDP, Wi-Fi	`config.h`	`onDecision()` dans le croquis

3.2 Ce qui se passe

1. Lancez la console sur le PC du casque.
2. Lancez votre client. Il attend d'être connecté ; un appareil UDP s'annonce chaque seconde.
3. Dans la console, sur l'accueil : « **Connecter une application** », choisissez la vôtre dans la liste, puis « **Connecter** ». Pour un appareil, « **Identifier** » fait clignoter ses LED : vous savez ainsi quelle ligne est la sienne.
4. Le client envoie ses réglages, puis démarre le mode. La console n'affiche plus que la **page de contrôle** : les électrodes, les réglages reçus, le décodage en direct et les dernières commandes.
5. Le mode **chauffe** 15 s, puis le SSVEP mesure 8 s de **repos** : ses premières décisions arrivent vers 23 s (40 s pour le Neuro). Pendant ce temps, la personne ne fixe aucune cible.
6. Les décisions arrivent dans votre gestionnaire. **`-1` veut dire « pas de décision »**, jamais la cible 0 : c'est une valeur normale et fréquente.
7. La connexion prend fin sur « **Déconnecter** », à la fermeture de la console, ou à l'arrêt de votre client. Le moteur arrête alors le mode que votre client a démarré, et le client dit pourquoi la connexion a pris fin.

Un **mode à modèle** (Motor Imagery, et par LSL P300, ErrP, c-VEP) attend une étape de plus : sur la page de contrôle, la personne choisit le modèle, puis clique « **Lancer le décodage** » (voir *Les modèles*).

Une seule application est connectée à la fois. Pendant la connexion, **votre application possède les réglages** : la console les affiche sans pouvoir les changer. Pour en changer un à la main, déconnectez d'abord l'application.

Sans console, avec le moteur seul, un client LSL du même PC est connecté tout de suite, sans clic. Le moteur seul n'écoute aucun appareil UDP. Pour le lancer, dans un terminal :

Programme	Commande
installé	`eeg-api-unicorn-cli.exe --serveur`, dans le dossier d'installation
depuis les sources	`python src/core/server.py`

Les deux acceptent les mêmes options, par exemple `--synthetic` pour ouvrir la séance sur le board de test.

3.3 Les clients LSL : Python, Unity, MATLAB

Les trois clients lisent les mêmes clés. Voici le fichier du client Python, `clients/python-lsl/config.json` :

```
{
  "app_name": "my-app",
  "mode": "ssvep",
  "params": {
    "targets_on": "screen",
    "target_names": ["left", "right", "up"]
  },
  "propose_refresh_hz": 60,
  "stop_mode_on_exit": true,
  "search_timeout_s": 30
}
```

Il demande un SSVEP sur un écran, à trois cibles nommées, et laisse le moteur choisir leurs fréquences pour un écran à 60 Hz.

Clé	Sens
`app_name`	le début du nom affiché par la console ; une partie aléatoire est ajoutée, pour que deux copies ne se confondent pas
`mode`	`ssvep`, `mi`, `neuro`, `p300`, `errp` ou `cvep` (le Brut n'a pas de décisions : lisez directement le flux `EEG_API_Unicorn_raw`)
`params`	les réglages du mode (partie 4) ; `{}` garde ceux de la console
`propose_refresh_hz`	SSVEP sur un écran : le rafraîchissement de l'écran ; le moteur propose des fréquences qui le divisent, et elles remplacent `freqs`. `0` : ne rien demander, obligatoire pour les autres modes et pour des fréquences que vous donnez vous-même. Unity : `-1` prend celui de l'écran de Unity
`stop_mode_on_exit`	à la fin, arrêter le mode que l'application a démarré
`search_timeout_s`	combien de secondes chercher un moteur avant d'abandonner

Python et MATLAB refusent une clé inconnue, avec la liste des clés connues ; Unity l'ignore sans rien dire : vérifiez l'orthographe des clés.

Client	Prérequis	Lancer	Le nom dans la console
Python	Python 3 et pylsl (`pip install pylsl`)	`python example.py`	`my-app-…`
Unity	`LSL.cs` et `lsl.dll`, copiés de LSL4Unity (voir le README du dossier)	ajoutez le composant **Eeg Api Client** à un GameObject, puis Play	`my-game-…`
MATLAB	liblsl-Matlab dans le chemin de MATLAB	`example`	`my-matlab-app-…`

À savoir :

- **SSVEP : votre application dessine les cibles**, une par fréquence, synchronisées sur l'écran. Elles clignotent dès le démarrage du mode, pendant la chauffe et le repos : le moteur mesure son repos avec les cibles qui clignotent. Les fréquences retenues sont dans la réponse au démarrage (`params.freqs`) ; le client Unity expose dans `Frequencies` celles que le moteur a proposées (vide si vous donnez vos propres `freqs`).
- **Unity : chaque Play crée une nouvelle application**, à reconnecter dans la console.
- **MATLAB** relit une liste d'un seul élément de `config.json` comme une valeur seule : donnez au moins deux éléments aux listes du mode.
- Arrêter le client (Ctrl+C, ou Stop dans Unity) arrête le mode qu'il a démarré.

3.4 Les clients UDP sur un ordinateur : Raspberry Pi, ROS 2

Le client Raspberry Pi n'utilise que la bibliothèque standard de Python 3.8 ou ultérieur. Son fichier, `clients/raspberry-pi/config.json` :

```
{
  "app_name": "my-device",
  "device_id": "",
  "device_port": 5011,
  "engine_host": "255.255.255.255",
  "engine_port": 5010,
  "mode": "ssvep",
  "params": {
    "targets_on": "physical",
    "freqs": [13, 15],
    "target_names": ["left", "right"]
  },
  "propose_refresh_hz": 0,
  "stop_mode_on_exit": true
}
```

Il demande un SSVEP sur deux cibles physiques : des LED à 13 et 15 Hz, que l'appareil fait clignoter lui-même. Les clés sont celles des clients LSL, plus le réseau :

Clé	Sens
`device_id`	l'identifiant affiché à côté du nom ; vide : la fin de l'adresse MAC
`device_port`	le port UDP où l'appareil écoute, et qu'il annonce (n'importe quel port libre)
`engine_host`	où partent les annonces : `255.255.255.255` (diffusion sur le réseau), ou l'adresse IP du PC de la console
`engine_port`	le port UDP de la console : `5010`
`mode`	`ssvep`, `mi` ou `neuro` : les modes servis par UDP

Lancez-le avec `python3 example.py`. « **Identifier** » dans la console appelle `on_identify()`.

ROS 2. Le paquet `clients/ros2/eeg_api_ros/` réutilise la même bibliothèque et publie chaque décision sur des topics. Son fichier, `config/params.yaml`, règle ses deux nœuds :

```
# The settings of the two nodes. This is the only file to edit.

eeg_api_bridge:
  ros__parameters:
    app_name: "my-robot"                # the name the console lists under "Connect an application"
    device_id: ""                       # empty: the end of this computer's MAC address
    device_port: 5011                   # the UDP port this node listens to, and announces
    engine_host: "255.255.255.255"      # where the hellos go: broadcast, or the console's IP address
    engine_port: 5010                   # the UDP port of the console
    mode: "ssvep"                       # ssvep, mi or neuro (the modes served over UDP)
    # The settings of the mode, as JSON (keys and ranges: docs/control.md). Here: two physical
    # targets (LEDs) blinking at 13 and 15 Hz.
    settings: '{"targets_on": "physical", "freqs": [13, 15], "target_names": ["forward", "turn"]}'
    propose_refresh_hz: 0.0             # > 0: SSVEP on a screen, ask the engine for frequencies
    stop_mode_on_exit: true

intent_to_twist:
  ros__parameters:
    linear_x: [0.1, 0.0]                # m/s for target 0, target 1...
    angular_z: [0.0, 0.5]               # rad/s for target 0, target 1...
    timeout_s: 1.0                      # no decision for this long: zero velocity
    cmd_vel_topic: "cmd_vel"
```

Les clés du pont (`eeg_api_bridge`) sont celles du Raspberry Pi, à une différence près : les réglages du mode s'écrivent en texte JSON, dans `settings`, car un paramètre ROS ne contient pas d'objet libre.

Topic	Type	Contenu
`eeg/target`	`std_msgs/Int32`	la cible décodée (SSVEP) ou la tâche (MI) ; `-1` : pas de décision
`eeg/confidence`	`std_msgs/Float32`	le score de la décision
`eeg/decision`	`std_msgs/String`	la décision entière, en JSON
`eeg/state`	`std_msgs/String`	`announcing`, `connected`, `decoding`, `disconnected` ou `failed`

Votre nœud s'abonne à `eeg/target`. Le nœud d'exemple `intent_to_twist` en fait une vitesse : la vitesse de la cible décodée, ou zéro sur `-1` et sans décision depuis `timeout_s`. Copiez `eeg_api_ros/` dans le dossier `src/` d'un espace de travail colcon, placez-vous à sa racine, puis :

```
colcon build --packages-select eeg_api_ros        # à refaire après chaque modification de params.yaml
source install/setup.bash
ros2 launch eeg_api_ros eeg_api.launch.py         # le pont et intent_to_twist
ros2 launch eeg_api_ros eeg_api.launch.py with_example:=false    # le pont seul
```

3.5 Les microcontrôleurs : ESP32, Arduino UNO R4 WiFi

L'ESP32 se connecte à la console par le Wi-Fi, sans ordinateur entre les deux. Pour un SSVEP, ses LED **sont** les cibles : elles clignotent grâce au matériel de la carte, à des fréquences que ni le Wi-Fi ni le programme ne décalent. Son fichier, `clients/esp32/esp32_client/config.h` :

```
// The settings of esp32_client: the only file to edit. Everything else works as it is.
// ⚠ This file holds your Wi-Fi password: do not publish it.

// --- The network -------------------------------------------------------------------------

// The Wi-Fi network the console's computer is on too: that computer's mobile hotspot, or the
// Wi-Fi of a router. At 2.4 GHz: the ESP32 does not see 5 GHz.
#define WIFI_SSID "network-name"
#define WIFI_PASS "password"

// Optional, a second network: the ESP32 takes the one it sees, the strongest if it sees both.
// #define WIFI_SSID_2 "network-name-2"
// #define WIFI_PASS_2 "password-2"

// Where the announcements go. "" = broadcast on the local network, which is usually enough.
// Otherwise the IP address of the console's computer (useful when the console never lists the
// device: a computer on several networks, a VPN). On the Windows mobile hotspot, the computer is
// at 192.168.137.1.
#define CONSOLE_IP ""

// --- The application ---------------------------------------------------------------------

// The name the console lists under "Connect an application", and the identifier shown next to
// it ("" = the end of the board's MAC address).
#define APP_NAME "esp32-app"
#define DEVICE_ID ""

// The UDP port this device listens to (any free port), and the console's (5010).
#define DEVICE_PORT 5011
#define ENGINE_PORT 5010

// The mode: "ssvep" (the LEDs ARE the targets: each one blinks at its frequency), "mi" (the LED
// of the decoded task lights up) or "neuro" (the values go to the serial monitor).
#define MODE "ssvep"

// One LED per target (SSVEP), or per task (MI, in the order of the model's tasks). Each LED has a
// resistor in series: (3.3 V - the LED's forward voltage) / the current, under about 20 mA per
// pin; 33 ohm suits a white or blue LED (about 3 V), about 100 ohm a red one.
#define TARGET_PINS {25, 26}
```

```
// SSVEP: the frequency (Hz) and the name of each target, in the order of TARGET_PINS. On LEDs any
// frequency works (no screen refresh rate to divide), at least 0.67 Hz apart.
#define TARGET_FREQS {13.0, 15.0}
#define TARGET_NAMES {"left", "right"}

// Other settings of the mode, a JSON object (docs/control.md, "Settings"). For SSVEP, targets_on,
// freqs and target_names are added from the lines above. Example: "{\"band_low\": 6}".
#define EXTRA_SETTINGS "{}"

// SSVEP: the chosen target stays steady for HOLD_MS, then decisions are ignored for REARM_MS more
// (the decoder's window still contains the pause).
#define HOLD_MS 3000
#define REARM_MS 1500

// The board's LED: on while a console is connected, blinking after a refusal. -1 = none.
#define STATUS_LED 2
```

⚠️ **Ce fichier contient le mot de passe du Wi-Fi : ne le publiez pas.** Le Wi-Fi doit être en 2,4 GHz : l'ESP32 ne voit pas le 5 GHz.

Ce que fait la carte, selon `MODE` :

`MODE`	Ce que fait la carte
`"ssvep"`	chaque LED clignote à sa fréquence ; la cible décodée reste allumée fixe `HOLD_MS`, puis toutes clignotent de nouveau
`"mi"`	la LED de la tâche décodée s'allume, les autres s'éteignent ; aucune sur `-1`
`"neuro"`	les valeurs s'affichent sur le moniteur série

Au plus 4 broches dans `TARGET_PINS` : la carte a quatre minuteurs matériels, un par cible. `TARGET_FREQS` et `TARGET_NAMES` ont autant d'éléments que `TARGET_PINS`, même en `"mi"`, où les fréquences ne servent pas : sinon, le croquis ne compile pas.

La LED de la carte (`STATUS_LED`) est allumée tant qu'une console est connectée, et clignote après un refus. Le moniteur série (115 200 bauds) dit ce que fait la carte, et la raison d'un refus.

Câblage. Une LED par cible, de la broche de `TARGET_PINS` vers GND, patte longue côté broche, avec une résistance en série : (3,3 V − la tension de la LED) / le courant, sous 20 mA environ par broche. 33 Ω conviennent à une LED blanche ou bleue (environ 3 V), une centaine d'ohms à une LED rouge.

Compiler et téléverser. Il faut le support des cartes ESP32 et la bibliothèque ArduinoJson 7. Dans l'IDE Arduino : installez « esp32 » (Espressif) dans le gestionnaire de cartes et « ArduinoJson » dans le gestionnaire de bibliothèques, ouvrez `esp32_client.ino`, choisissez la carte (« DOIT ESP32 DEVKIT V1 » pour un ESP32 DevKit V1), puis Téléverser. Avec arduino-cli :

```
arduino-cli core install esp32:esp32
arduino-cli lib install ArduinoJson
arduino-cli compile --fqbn esp32:esp32:esp32doit-devkit-v1 esp32_client
arduino-cli upload --fqbn esp32:esp32:esp32doit-devkit-v1 -p COM5 esp32_client
```

(`COM5` est un exemple : `arduino-cli board list` donne le port de la carte.)

Arduino UNO R4 WiFi. Le client `clients/arduino-wifi/` utilise la même bibliothèque, avec un `config.h` presque identique : `MODE` vaut `"mi"` ou `"neuro"`, une LED par tâche dans `TASK_PINS` (environ 220 Ω en 5 V), et les réglages du mode dans `SETTINGS`. Pour un SSVEP sur des LED, prenez un ESP32 : lui seul fait clignoter les cibles par le matériel. Carte : « Arduino UNO R4 WiFi », support « Arduino UNO R4 Boards » (`arduino:renesas_uno:unor4wifi`).

4 Les réglages du mode

Le client envoie les réglages avant de démarrer le mode. Ils s'écrivent dans `params` (clients LSL et Raspberry Pi), `settings` (ROS 2), `EXTRA_SETTINGS` (ESP32) ou `SETTINGS` (UNO R4).

Mode	Réglage	Sens
`ssvep`	`targets_on`	`"screen"` (des cibles à l'écran) ou `"physical"` (des LED)
	`freqs`	une fréquence par cible, en Hz (2 à 10 cibles)
	`target_names`	un nom par fréquence, dans le même ordre (facultatif)
	`refresh_hz`	le rafraîchissement de l'écran du stimulus (écran seulement)
	`alpha_hz`	le pic alpha de la personne, mesuré par « Vérifier le casque »
`mi`, `p300`, `errp`, `cvep`	`model`	le nom de fichier du modèle : avec la console, une simple suggestion
`p300`, `errp`, `cvep`	`stream_in`	le nom du flux de marqueurs que publie votre application

La liste complète, avec les bornes de chaque réglage, est dans `docs/control.md`.

- **Le moteur est le seul juge.** Un réglage refusé l'est avec sa raison, en anglais, et le client l'affiche. N'écrivez pas les règles du moteur dans votre application : lisez ses refus.
- **SSVEP sur un écran** : une cible ne clignote qu'à une fréquence qui divise le rafraîchissement (à 60 Hz : 30, 20, 15, 12, 10, 8,571…), et le moteur refuse les autres. N'écrivez pas les fréquences à la main : donnez `propose_refresh_hz`, et le client demande au moteur un jeu de fréquences adapté à l'écran et au pic alpha.
- **SSVEP sur des LED** (`"targets_on": "physical"`) : toute fréquence de la bande du filtre convient, à condition que deux cibles soient écartées d'au moins 0,67 Hz.
- Un réglage changé pendant que le mode tourne relance souvent la chauffe et le repos.

5 Les modèles

Quatre modes ne décodent rien sans modèle : Motor Imagery, P300, ErrP et c-VEP. Un modèle s'apprend sur **la personne** qui porte le casque, avec **ce** casque, si possible **pendant la même séance**. Le modèle d'une autre personne ne provoque aucune erreur : il donne des réponses plausibles, mais fausses.

Mode	Ce que le modèle apprend	Durée de l'entraînement
Motor Imagery	les tâches imaginées (2 à 5, par défaut LEFT et RIGHT)	indiquée par la page, selon les tâches et les essais
P300	la réponse aux flashs de la cible choisie (3 à 10 cibles, 6 par défaut)	environ 2,2 min à 6 cibles
ErrP	la réaction à une erreur de la machine	environ 5,7 min
c-VEP	la réponse au code clignotant (2 à 10 cibles, 6 par défaut)	environ 3,1 min à 6 cibles et 60 Hz

Le SSVEP et le Neuro n'ont pas de modèle : ils mesurent un repos à chaque démarrage.

5.1 Entraîner un modèle

L'entraînement se fait **uniquement dans la console** : aucune commande ne permet à une application d'entraîner un modèle.

1. Passez d'abord « **Vérifier le casque** ».
2. Ouvrez la page du mode : sa tuile, puis « **Ouvrir** ». La page a ses étapes numérotées : **1. Régler**, **2. Entraîner**, **3. Tester**. Cliquez sur « **Entraîner** ».
3. Choisissez ce que le modèle va apprendre, **avant** de commencer : il gardera ces choix.

Mode	À choisir
Motor Imagery	le nombre de tâches et leurs noms (LEFT, RIGHT, ou FEET, HANDS…), la mise en route (1 à 3 s laissées pour lancer l'imagination avant l'enregistrement), le nombre d'essais par tâche
P300, c-VEP	le nombre de cibles et leurs noms ; pour le c-VEP, le rafraîchissement de l'écran du stimulus (60 Hz par défaut)
les quatre	la bande du filtre : la valeur par défaut convient le plus souvent

   ⚠️ Sans tâche de repos, un modèle de Motor Imagery désigne **toujours** une tâche : seul son seuil de probabilité (`prob_min`) le fait taire, en publiant `-1` quand il n'est pas assez sûr. Si votre application doit savoir quand la personne ne fait rien, ajoutez une tâche REST.
4. Cliquez sur « **Commencer** ». La console contrôle d'abord la liaison avec le casque et montre le contact de chaque électrode. Si le signal d'une seule des huit électrodes sort de [0,5 ; 500] µV, ou si la référence est décollée, elle refuse et dit pourquoi : corrigez, puis recommencez. Sinon, cliquez sur « **Commencer l'entraînement** ».
5. Suivez les consignes. Pour le Motor Imagery, la console les affiche elle-même. Pour le P300, l'ErrP et le c-VEP, elle ouvre une **fenêtre de stimulus** sur l'écran principal (pour le c-VEP, un écran noir d'une seconde au début et à la fin est normal).
6. À la fin, la console affiche le **résultat** : un niveau (vert, orange ou rouge), un chiffre, et le hasard auquel le comparer. Deux boutons :
   - « **Enregistrer le modèle** » : le modèle est écrit dans le dossier des modèles, sous un nom horodaté (par exemple `p300_model_20261029_101500_6targets.joblib`) ;
   - « **Refaire** » : rien n'est gardé.

Le dossier des modèles est `Documents\EEG API Unicorn\modeles\` pour le programme installé, et `data\` à la racine du dépôt depuis les sources. **Rien n'y est écrit avant le clic sur « Enregistrer le modèle ».** Le c-VEP enregistre deux modèles par séance, un par algorithme (eCCA et rCCA), aux performances comparables.

L'ordre d'une séance est donc : **entraîner, enregistrer, tester, puis connecter**. Tant qu'un résultat attend « Enregistrer le modèle » ou « Refaire », la console refuse de connecter une application ; pendant une connexion, aucun entraînement n'est possible.

5.2 Tester un modèle

Un modèle enregistré n'est pas forcément un bon modèle. Le bloc **« Tester »** de la page du mode le vérifie :

Bouton	Ce qu'il fait
« Tester sur une session »	rejoue le protocole de l'entraînement, mais le moteur **décide** au lieu d'apprendre, et rend un score comparé au hasard
« Tester librement » (c-VEP, SSVEP, P300, Motor Imagery)	fait tourner le mode et entoure en direct ce qu'il décode, sans score

Pour le SSVEP, qui n'a pas de modèle, « Tester sur une session » est une séance guidée ; il est désactivé avec des cibles physiques, car il désigne des disques à l'écran.

⚠️ Deux tests donnent souvent un résultat rouge même quand tout fonctionne : celui du Motor Imagery, qui demanderait beaucoup plus d'essais pour écarter le hasard, et celui du c-VEP, dont le vert est exigeant à la longueur par défaut. Ne concluez pas sur un seul test.

5.3 Les modèles pendant une connexion

Quand une application est connectée, **c'est la personne devant la console qui choisit le modèle**, jamais l'application :

1. Votre client démarre un mode à modèle. Le moteur répond que le mode **attend** (`"deferred": true`).
2. Sur la page de contrôle, la personne coche un modèle, par son nom de fichier.
3. Elle clique sur **« Lancer le décodage »**. Le mode démarre, et les décisions arrivent.

- Le `model` que donne votre application n'est qu'une **suggestion** : la console le coche d'avance s'il existe, sans jamais l'appliquer d'office.
- **S'il n'existe aucun modèle entraîné** pour ce mode sur le PC de la console, le démarrage est refusé : déconnectez l'application, entraînez un modèle, puis reconnectez-la.
- Si la personne change de modèle (**« Relancer avec ce modèle »**), le flux des décisions est recréé ; les clients prêts à l'emploi le rouvrent d'eux-mêmes.

5.4 Ce que le modèle impose

Le modèle **porte** ses paramètres : le nombre de cibles ou de tâches, leurs noms, leur ordre, le rafraîchissement (c-VEP), la bande du filtre. Ne les écrivez pas en dur dans votre application :

- **par LSL**, ils sont dans la description du flux des décisions, sous l'élément `decoding` : `n_targets` (P300, c-VEP), `frequencies_hz` (SSVEP), `classes` (les tâches du MI), `refresh` (c-VEP) et `target_names` quand des noms sont définis. Chaque valeur est un texte, et une liste y est jointe par des virgules ;
- **par UDP**, un appareil ne reçoit pas le nom des tâches du Motor Imagery, seulement leur numéro. Leur ordre est celui choisi à l'entraînement, affiché sur la page de contrôle : câblez les LED de `TASK_PINS` (ou `TARGET_PINS`) dans cet ordre.

6 Afficher le stimulus : P300, ErrP, c-VEP

Pour ces trois modes, **votre application affiche le stimulus** et dit au moteur, à l'instant exact, ce qu'elle vient d'afficher. Elle le fait par un flux LSL de **marqueurs**, nommé `EEG_API_Unicorn_stim` par défaut. Sans marqueurs, le mode ne décode rien. Les clients prêts à l'emploi s'occupent de la connexion ; les marqueurs restent à votre application. Ils ne passent pas par UDP, car ils sont datés dans l'horloge de LSL.

Mode	Marqueur	Quand l'envoyer
P300	`{"mode": "p300", "event": "flash",` , puis `"target": 3}` `{"mode": "p300", "event": "round_end"}`	à chaque flash, puis à la fin de chaque manche (chaque cible flashe 2 à 8 fois par manche)
ErrP	`{"mode": "errp", "event": "feedback"}`	à chaque résultat affiché, juste ou non
c-VEP	`{"mode": "cvep", "event": "cycle", "refresh": 60.0}`	chaque fois que le code repart de sa première image, environ une fois par seconde

Trois règles font le décodage :

1. **Datez chaque marqueur juste APRÈS l'affichage de l'image** (après le `flip`), jamais avant. Quarante millisecondes d'avance suffisent pour que le moteur décode du bruit, sans aucune erreur.
2. **Affichez le stimulus que le modèle a appris** : pour le P300, des cibles numérotées comme celles du modèle (`target` de 0 à `n_targets − 1`) ; pour le c-VEP, le même code, aux mêmes décalages et au même rafraîchissement (ceux de `src/core/cvep_code.py`).
3. **N'envoyez jamais la réponse** pendant le décodage : pas de champ `error` sur le marqueur de l'ErrP, pas de cible sur celui du c-VEP.

Règles propres à chaque mode :

- **P300** : laissez environ 2,5 s entre un `round_end` et le flash suivant, le temps que le regard change de cible. Espacez les flashs de 150 ms, comme à l'entraînement. Envoyez toujours le `round_end`, même si la manche est interrompue : sans lui, elle est abandonnée au bout de 10 s.
- **P300 et ErrP** : les marqueurs envoyés pendant la chauffe (15 s ; 23 s pour l'ErrP, repos compris) sont jetés. Commencez quand la phase vaut `decoding`.
- **c-VEP** : `refresh` est un nombre, égal à celui du modèle à 1 Hz près ; sinon les marqueurs sont refusés et le mode publie `-1` sans fin. Envoyez un marqueur à chaque cycle : après 3 cycles sans marqueur, le mode cesse de décoder. Faites clignoter dès le démarrage du mode, chauffe comprise : le c-VEP garde ces marqueurs.

Si plusieurs émetteurs peuvent publier sur le même réseau, donnez au vôtre un nom unique (par exemple `EEG_API_Unicorn_stim_mon-jeu`) et indiquez-le dans le réglage `stream_in`, **avant** de démarrer le mode. Si un autre mode à marqueurs tourne déjà sur un autre flux, arrêtez-le d'abord : le mode démarré écouterait le sien.

Le protocole complet est dans `docs/markers.md`. Les fenêtres de `src/stimulus/` (`p300.py`, `errp.py`, `cvep.py`) en sont les implémentations de référence.

7 Ce que reçoit l'application

7.1 Les décisions, mode par mode

Une décision contient les voies du flux du mode, par leur nom :

Mode	Voies	Cadence
SSVEP	`target_index`, `freq_hz`, `confidence`, un `score_<f>Hz` par fréquence	environ 5 par seconde
Motor Imagery	`intent_index`, `confidence`, `p_0` … `p_<n−1>` (une probabilité par tâche)	environ 5 par seconde
Neuro	`charge`, `somnolence`, `engagement`, `artifact`	environ 5 par seconde
P300	`target_index`, `confidence`, `n_flashes`, `score_0` …	une par manche
ErrP	`error`, `score`, `threshold`, `artifact`	une par résultat affiché
c-VEP	`target_index`, `confidence`, `score_0` …, `corr_min`, `margin`	environ 5 par seconde

- **`-1` sur `target_index` ou `intent_index` veut dire « pas de décision »**, jamais la cible 0. Un mode qui n'est pas sûr de lui publie `-1` : c'est souvent la valeur la plus fréquente. Traitez-le comme « ne rien faire ».
- **ErrP** : `error` vaut `1` (erreur), `0` (pas d'erreur) ou `-1` (pas de verdict, qui n'est pas un `0`).
- **Un flux muet** veut dire autre chose : le mode chauffe, mesure son repos ou est arrêté ; ou bien le P300 et l'ErrP ne reçoivent pas de marqueurs.
- **Par UDP**, chaque voie arrive en nombre : `target_index` vaut `1.0`, et `-1.0` veut dire « pas de décision ».
- Les valeurs du **Neuro** sont relatives au repos mesuré au démarrage, pour cette personne et cette séance : elles ne se comparent ni entre personnes, ni entre séances. Elles peuvent valoir `-1` sans que cela veuille dire « pas de décision ».
- **Chaque mode a son échelle** (`decision_scale`, dans la description du flux) : le SSVEP décide sur un `z`, le MI sur une probabilité, le P300 et l'ErrP sur un `logodds`, le c-VEP sur une corrélation. Un seuil propre à un mode n'a pas de sens pour un autre.
- **P300** : `confidence` est un log-odds, le plus souvent négatif. Ne filtrez pas sur `confidence > 0` : vous jetteriez toutes les bonnes réponses. Lisez `target_index`.
- Pendant un **test** lancé dans la console, le flux du mode porte les décisions du test : votre application les reçoit aussi.

Pour voir les flux tels qu'une application les reçoit, ouvrez **« Ce que voit ton application »**, en bas de l'accueil de la console.

7.2 L'état du moteur

Le flux `EEG_API_Unicorn_status`, republié toutes les 2 s, décrit le moteur :

Champ	Contenu
`modes`	la liste des modes en marche, par exemple `["raw", "ssvep"]`
`phase`	`warmup` (la chauffe), `baseline` (le repos : ne fixer aucune cible), `decoding`, `calibrating` (un entraînement) ou `streaming` ; avec plusieurs modes, celle du moins avancé
`instruction`	la consigne pendant la chauffe et le repos
`marqueurs`	ce que le moteur entend des marqueurs : `connecte` (faux : il n'entend aucun flux de marqueurs), `perdus`, `futurs`, `illisibles`, `inlet_erreurs`. Le premier endroit où regarder quand un P300, un ErrP ou un c-VEP ne décode rien

Par UDP, le moteur pousse un `status` plus court, limité aux modes que l'appareil a demandés. ⚠️ `modes` y est un dictionnaire mode → phase (`{"ssvep": "decoding"}`), et `phase` peut aussi valoir `idle` ou `deferred` (un mode à modèle attend le choix de son modèle).

8 Écrire son propre client

Cette partie sert à écrire un client sur une plateforme qui n'en a pas. Les deux bibliothèques `clients/python-lsl/eeg_api_lsl.py` (LSL) et `clients/raspberry-pi/eeg_api_udp.py` (UDP) en sont les implémentations de référence, et `docs/control.md` le contrat complet.

8.1 Les commandes

Il y a quatre commandes, et aucune autre. Elles sont les mêmes par LSL et par UDP.

Commande	Effet
`set_params`	change des réglages. Sur un mode arrêté, ils sont **vérifiés puis retenus** pour le prochain démarrage (`"deferred": true`) ; sur un mode en marche, ils s'appliquent
`start_mode`	démarre le mode avec ses réglages retenus ; déjà en marche : `"already_running": true`
`stop_mode`	arrête le mode ; déjà arrêté : `"already_stopped": true`
`propose_params`	demande au moteur de **calculer** un réglage sans l'appliquer : les fréquences du SSVEP pour un écran donné

Une commande est un objet JSON, envoyé en un seul texte, avec un **`ref`** unique :

```
{"cmd": "set_params", "mode": "ssvep", "params": {"targets_on": "physical",
 "freqs": [13, 15], "target_names": ["left", "right"]}, "ref": "mon-jeu-1"}
```

La réponse recopie ce `ref`, et liste tous les réglages du mode (abrégés ici) :

```
{"ref": "mon-jeu-1", "cmd": "set_params", "mode": "ssvep", "accepted": true,
 "deferred": true, "warning": null, "params": {"targets_on": "physical", "freqs": [13.0, 15.0]}}
```

- **L'ordre normal : régler, puis démarrer.** Envoyez `set_params`, attendez qu'il soit accepté, puis envoyez `start_mode`.
- **Une commande se renvoie sans risque.** Sans réponse au bout de quelques secondes, la commande ou sa réponse a été perdue : renvoyez-la, avec le même `ref`.
- **Un refus** porte `"accepted": false` et une `reason` en anglais, qui dit la cause et souvent la correction. Affichez-la.
- **Un démarrage accepté peut encore échouer** un peu plus tard (un fichier de modèle illisible) : un second message arrive, avec `"event": "failed"` et le `ref` du `start_mode`.
- **SSVEP sur un écran** : envoyez `propose_params` avec `{"refresh_hz": 60}`. La réponse donne `key` (`freqs`) et `value` (les fréquences). Appliquez-les ensemble par `set_params` : `{"refresh_hz": 60, "freqs": <value>}`. Un `refresh_hz` envoyé seul est refusé dès que les fréquences retenues ne le divisent pas.
- Au-delà d'environ 20 commandes par seconde, les suivantes sont refusées.

8.2 Par LSL

Flux	Sens	Créé par	Un échantillon
`EEG_API_Unicorn_control`	application → moteur	**votre application**	une commande, en texte JSON
`EEG_API_Unicorn_replies`	moteur → application	le moteur	une réponse, en texte JSON
`EEG_API_Unicorn_decoded_<mode>`	moteur → application	le moteur	une décision

Les deux premiers sont de type `Markers`, à une voie, au format `string`, à débit irrégulier.

Dans l'ordre :

1. **Ouvrez tous les flux `EEG_API_Unicorn_replies` du réseau**, avant toute commande : une réponse publiée avant leur ouverture est perdue.
2. **Créez le flux de commandes.** Son `source_id` est le nom que la console affiche dans sa liste : choisissez-le explicite, en ASCII simple, et unique si plusieurs copies peuvent tourner. Sans ce nom lisible, le message de déconnexion ne pourrait pas vous désigner.

3. **Attendez que le moteur le lise** (`wait_for_consumers`) : une commande envoyée avant est perdue. Avec la console, cette attente dure jusqu'au clic sur « Connecter » : ne l'abrégez pas, et dites à l'utilisateur ce qui est attendu.
4. **Envoyez les commandes**, chacune avec un `ref` unique : le flux de réponses est partagé par toutes les applications du réseau, et le `ref` désigne les vôtres.
5. **Ouvrez le flux des décisions de CE moteur.** Tous les moteurs publient sous le même nom ; ils se distinguent par le `source_id`, `EEG_API_Unicorn_decoded_<mode>@<identifiant du moteur>`. L'identifiant est celui du flux de réponses qui vous a répondu.
6. **Lisez les décisions.** Si le moteur recrée le flux (nouveaux réglages, nouveau modèle), rouvrez-le.
7. **À la fin**, envoyez `stop_mode`, puis fermez le flux de commandes : la connexion prend fin avec lui.

La déconnexion arrive par un message du flux de réponses, qui ne répond à aucune commande :

```
{"ref": null, "event": "disconnected", "cause": "console", "app": "mon-jeu",
 "stopped": ["ssvep"], "reason": "Disconnected in the console: ..."}
```

Champ	Sens
`cause`	`"console"` : la personne a cliqué sur « Déconnecter » ; `"engine_stopped"` : le moteur s'est arrêté
`app`	le `source_id` du flux de commandes concerné : ne retenez que le message qui porte le vôtre
`stopped`	les modes arrêtés par le moteur
`reason`	la même chose, en anglais, pour les personnes : votre programme ne doit pas dépendre de sa formulation

Pour continuer après une déconnexion : fermez le flux de commandes, créez-en un nouveau, attendez d'être reconnecté, puis renvoyez les réglages et le `start_mode`. ⚠️ Ne comptez pas sur `have_consumers()` pour détecter le départ du moteur : LSL ne le remarque qu'au prochain envoi.

8.3 Par UDP

La console écoute l'UDP sur le **port 5010**. Six règles :

1. **Un datagramme = un objet JSON, en UTF-8, qui porte `"v": 1`**, de 8 Ko au plus. Sans `"v": 1`, il est ignoré sans réponse.
2. **Envoyez tout depuis le port annoncé**, avec une seule socket liée à ce port : le moteur reconnaît l'appareil à l'adresse d'où partent ses datagrammes. Un datagramme d'un autre port est ignoré sans réponse.
3. Une fois connecté, **répondez à l'adresse d'où vient le `connect`** : c'est celle du moteur.
4. Chaque message poussé par le moteur porte un `seq` croissant : ignorez celui dont le `seq` n'est pas supérieur au dernier reçu, et repartez de 0 à chaque `connect`.
5. **UDP peut perdre un datagramme** : renvoyez une commande restée 2 s sans réponse. Lisez avec un tampon de 8 Ko : les réponses dépassent souvent 255 octets.
6. **Une fois connecté, envoyez `{"v": 1, "alive": true}` chaque seconde.** Après 5 s sans rien de valide de l'appareil, le moteur le considère comme parti et arrête ce qu'il a démarré.

Étape	Émetteur	Datagramme
S'annoncer, chaque seconde, tant qu'on n'est pas connecté	appareil → `255.255.255.255:5010`, ou l'IP de la console	`{"v": 1, "hello": "EEG_API_Unicorn", "app": "mon-jeu", "id": "AB:CD", "port": 5011}`
Identifier (un clic, facultatif)	moteur → appareil	`{"v": 1, "identify": true}` : faites clignoter une LED
Connecter (un clic)	moteur → appareil	`{"v": 1, "connect": "<moteur>", "host": "<nom du PC>"}`
	appareil → moteur	`{"v": 1, "connected": true}` ; déjà connecté ailleurs : `{"v": 1, "connected": false, "busy": "<autre PC>"}`
Commandes	appareil → moteur	`{"v": 1, "cmd": "start_mode", "mode": "ssvep", "ref": 1}`
	moteur → appareil	la même réponse que par LSL, plus `"v"` et `"seq"`
Pendant la connexion	moteur → appareil	`{"v": 1, "seq": n, "status": {...}}` à chaque changement et chaque seconde ; `{"v": 1, "seq": n, "decision": {...}}` à chaque décision
	les deux	`{"v": 1, "alive": true}` chaque seconde
Fin	appareil → moteur	`{"v": 1, "bye": true}`
	moteur → appareil	`{"v": 1, "seq": n, "event": "disconnected", ...}`

- Le mot `"EEG_API_Unicorn"` s'écrit exactement ainsi, et `port` est un entier.
- Le moteur envoie `connect` **une seule fois** et attend 3 s : sans réponse, la console dit que l'appareil ne répond pas, et il faut cliquer de nouveau.
- Après un `disconnected`, comme après 5 s sans rien du moteur, l'appareil se considère comme déconnecté et recommence à s'annoncer.
- Les messages `identify`, `connect` et `alive` du moteur ne portent pas de `seq`.
- Sur un ESP32, un datagramme de plus d'environ 1 470 octets n'arrive jamais ; les messages des modes servis par UDP sont bien plus courts.

9 Dépannage

Symptôme	Cause probable
l'application n'apparaît pas dans la liste de la console	elle n'est pas sur le même réseau, ou un pare-feu la bloque (autorisez le programme, réseaux publics compris) ; certains Wi-Fi isolent les appareils : utilisez le point d'accès mobile du PC de la console
un appareil UDP n'apparaît pas	sa diffusion n'atteint pas la console (un PC sur plusieurs réseaux, un VPN) : mettez l'adresse IP de la console dans `engine_host` ou `CONSOLE_IP` ; ou c'est le moteur seul qui tourne, pas la console
la console dit que l'appareil ne répond pas	il n'a pas répondu au `connect` en 3 s, ou il a répondu depuis un autre port que celui de son annonce
la console dit que l'appareil ne donne plus de nouvelles	rien de valide reçu depuis 5 s : son `alive` omet `"v": 1`, ou part d'un autre port que celui de son annonce
la console dit que l'appareil est déjà connecté	il a répondu `"busy"` : une autre console le tient ; déconnectez-le d'abord là-bas
le client attend d'être connecté, sans fin	personne n'a cliqué sur « Connecter » ; avec le moteur seul, l'application ne tourne pas sur le même PC
`"accepted": false`	la `reason` dit la cause : un réglage hors bornes, un mode inconnu, aucun modèle entraîné…
le mode attend (`"deferred": true`)	un mode à modèle : la personne choisit le modèle, puis clique sur « Lancer le décodage »
un mode démarré ne publie rien	il chauffe (15 s) puis mesure son repos (environ 23 s au total pour le SSVEP, 40 s pour le Neuro) ; le P300 et l'ErrP ne décodent rien sans marqueurs
des décisions, mais au hasard	le contact des électrodes (relancez « Vérifier le casque »), le modèle d'une autre personne ou d'une autre séance, un stimulus différent de celui du modèle, ou des fréquences SSVEP différentes de celles qui clignotent
le mode s'arrête quand l'application redémarre	elle a recréé son flux de commandes : c'est une nouvelle application, à reconnecter
la console refuse de connecter l'application	un résultat d'entraînement attend « Enregistrer le modèle » ou « Refaire », ou un essai est en cours : terminez-le d'abord
votre propre client n'a pas de réponse	la commande est partie avant que le moteur lise le flux, ou les réponses ont été ouvertes trop tard : renvoyez-la
on ne sait pas ce qui circule sur le réseau	ouvrez « Ce que voit ton application » dans la console : chaque flux et ses valeurs, tels qu'une application les reçoit

10 Limites

- **Une seule application** est connectée à la fois à une console.
- **Réseau local uniquement** : la découverte LSL et la diffusion UDP ne traversent pas les routeurs. Les échanges ne sont ni chiffrés ni authentifiés : seul le clic sur « Connecter » autorise une application.
- **Par UDP**, seuls le SSVEP, le Motor Imagery et le Neuro sont servis ; le P300, l'ErrP, le c-VEP et le Brut sont refusés, avec leur raison. Le Motor Imagery n'y transmet pas le nom des tâches.
- **Les performances du décodage** dépendent de la personne, du contact des électrodes et de la séance : mesurez-les avec « Tester » avant de vous y fier.